Week 1:

I followed the teacher's code along, recreating the stack data structure and it's help functions.
then on Thursday I:

- Created a couple new Debug Tasks.
- Changed the Stack to be a Que
- Made it so on Awake(), some test/debug tasks get added to the que
- Made some condition checks for when the player interacts with the Computer
- Reformatted the code to go back to the standard I'm used to.

Friday:

Did a code review and implemented a stack data structure. (so now I have both), but I'll be using a que.

Week 2:

Monday:

I reworked the `readFromFile` task so it now also parses the data, converting it into a appropriate Array.
Worth mentioning is that JSON can only take `string`, `Bool` and `int/float` as datatypes. For other dataTypes a custom class has to be made to convert the `JSON` string data to the dataType you desire.

This `ReadFromFile` Task also sends it's array of `Employees` to a `Data Scriptable object`. Which can later get accessed from *anywhere*
(given it has a reference to that scriptable object instance).

Wednesday

I quickly made a insertion sort algo following Jo's example

Thursday

I continued to work on implementing new sorting algos.
I added a Bubble Sort.

Then started work on a Merge Sort algo, but that got complicated...
I wasn't quite sure as to how/when the combination of the sub arrays should/do happen.

Friday

Checked my work. I passed then went home

Week 3:

Monday

Worked on and *finished* my Merge Sort implementation.
(I *just* had to look into when the recursion reaches a collection size of 1)

Then I worked on Quick Sort.

Tuesday

took this day to do house-keeping/etc (no school work)

Wednesday

I worked on Quick Sort, trying to understand it more.
But there's still some ?.

I'll read up on it a bit more, for example on [here](Quick Sort - GeeksforGeeks)

(Quick Sort got confusing. As I didnt want to define multiple variables and just "find"/calculate them via the median.)

Finally I decided that I should convert to Merge Sort.
I also converted the Merge Sort I had so it can handle the Employee and Employee[] data types instead of int, and int[].
I also tested it and made sure it works, which it does.

Thursday

Went over Week 4's work (searching)

I both, started work on the Searching task *and* finished the implementation of it.
So the Task can now "search" a sorted collection for a *exact* match of a date.
It returns the first instance it can find. (*IF* a exact match is found that is. Otherwise it returns null)

Friday

I started by creating a new task following these assesment instructions:

```
Week 3 Assessment.
1. Create a new Task, name it SecretTask
2. Inside of the task, create a List and populate it with the values 8 2 1 3
9
3. Write a small function that prints out the List with spaces between the
values.
4. Write a comparison function that takes in a List, along with an index.
   The function should compare the value at index and index + 1's position.
   if the value at index + 1 is lower than the value at index, the values
should be swapped.
5. Make use of the above function to iterate through each pair in the list,
and compare them.
   After each comparison we should print the state of the list. Figure out
if you can add something to the print function to make it useful and
readable.
6. Write an outer Loop that does 5. 'enough' times.
7. Rewrite the code so that it can handle a list of arbitrary length.
```

(This "secretTask" works similarly to bubble sort.)

I Show-cased my work and I passed.

Week 4:

Monday

Added the fullEmployee.JSON to the project, and made it so the tasks use this data instead of the testEmployee.JSON

I reworked the search Task so it now returns a collection of employees within a specified range not just a singular employee with a exact match on that date.
(The search task can and does do both the SingularBinarySearch and RangedBinarySearch)

Also added a couple in-line code to compare the time taken to execute between singular and ranged search

Wednesday

Tested the search function with different ranges.
I found that I need to make sure the median was even by seeing `if (median % 2 > 0) { median++; }`
So that I could always split the collection in halves.
This also meant I had to check for `null` elements. If a element is null then that means we are at the end of this collection.

I also found a case where I used to only check if the length was less than 1, I belive I instead need to check if this singular element is withing the specified search range.
(IF not return null, indicating that the element doesnt exist)

Thursday

I followed along in the code along with Jo for Binary Search Trees

I found that the collection I am returning after having searched isnt sorted. ... BUT the assignment doesnt specify that this return collection has to be (sorted). ;D

I implemented a Binary Search Tree (BST) and a search function utalising this new data structure.

Then I felt it was necessary to move the benchmarking code into the already created benchmarking script, since the assignment specifies this. Then I made it so I use that script and it's functions to benchmark instead of the "in-line" code I used to have.
(This made the results *much* slower though...)

Friday

Did a paper walkthrough for this times assessment according to these instructions:

Procedure [edit]

Given an array A of n elements with values or records $A_0, A_1, A_2, \ldots, A_{n-1}$ sorted such that $A_0 \leq A_1 \leq A_2 \leq \cdots \leq A_{n-1}$, and target value T, the following subroutine uses binary search to find the index of T in A.[7]

1. Set L to 0 and R to $n-1$.
2. If $L > R$, the search terminates as unsuccessful.
3. Set m (the position of the middle element) to L plus the floor of $\frac{R-L}{2}$, which is the greatest integer less than or equal to $\frac{R-L}{2}$.
4. If $A_m < T$, set L to $m+1$ and go to step 2.
5. If $A_m > T$, set R to $m-1$ and go to step 2.
6. Now $A_m = T$, the search is done; return m.

```
Given this procedure showcase two full examples in your notebook when n =
10, and n = 50. Choose Ax and T in such a way so the procedure runs through
most iterations possible.
2. Roughly, how many more iterations does this procedure run, if n = 80000,
as compared to when n = 10000, assuming worst case.
3. What is the mathematical function one can use to deduce the number of
iterations this procedure will take, given a specific n?
```

(I made a flowchart of how the procedure works:)
Data Structures - Friday Assessment task - FlowChart.canvas

I'd like to refactor this binary search so it utalises a minimum- and maximum-index, instead of creating new arrays `Employee[]` each time it splits the collection...

Week 5:

Monday

I went back to my Binary Search and refactored it to utalise minimum- and maximum indexes. Instead of creating arrays `Employee[]` each time we exclude a part of the collection.

Thursday

I went through my notes on the lectures and these report notes.
Then re-wrote it so it more coherently and consisely answers the assignment's questions.